

Cryptocurrency Price Tracker

1. Project Title

Cryptocurrency Price Tracker using Python and Selenium

2. Submitted By

Name: Nithish Kumar V

3. Objective

The objective of this project is to develop a Python-based automation tool that extracts real-time cryptocurrency data such as coin name, current price, 24-hour change, and market capitalization from CoinMarketCap and stores it in a structured CSV format.

4. Project Description

The Cryptocurrency Price Tracker is a Selenium-based web scraping tool that automates browser interaction to collect live cryptocurrency data. Since CoinMarketCap loads data dynamically using JavaScript, Selenium is used to render the page and extract accurate information.

The script retrieves the top 10 cryptocurrencies and records their details along with a timestamp. The extracted data is then saved into a CSV file for further analysis, monitoring, or integration with dashboards.

5. Technologies Used

- **Python** – Programming language
 - **Selenium** – For browser automation and dynamic scraping
 - **pandas** – For data handling and CSV export
 - **webdriver_manager** – For automatic ChromeDriver setup
 - **time module** – For delays
 - **datetime module** – For timestamp logging
-

6. Methodology

1. Initialize Chrome WebDriver using Selenium
2. Open the CoinMarketCap website
3. Wait for the cryptocurrency table to load
4. Scroll the page to ensure all data is rendered

5. Extract top 10 cryptocurrency details:
 - Name
 - Current Price
 - 24-hour Change
 - Market Capitalization
 6. Add timestamp to each record
 7. Store data in a list of dictionaries
 8. Convert data into a pandas DataFrame
 9. Export the data to a CSV file
-

7. Code Explanation

- Selenium WebDriver is used to control the browser
 - Headless mode allows execution without opening a browser window
 - WebDriverWait ensures the page is fully loaded before scraping
 - XPath selectors are used to locate rows and columns in the table
 - Data is extracted from specific column indices
 - Each record is stored as a dictionary and appended to a list
 - pandas is used to convert the list into a DataFrame
 - The DataFrame is exported to a CSV file
-

8. Output

The output of the project is a CSV file named **crypto_prices.csv** containing the following columns:

- Name
- Current Price
- 24h Change
- Market Capitalization
- Timestamp

This file can be opened in Excel for analysis.

9. Advantages

- Provides real-time cryptocurrency data

- Automates data collection process
- Enables historical tracking using timestamps
- Can be used for data analysis and dashboards

10. Limitations

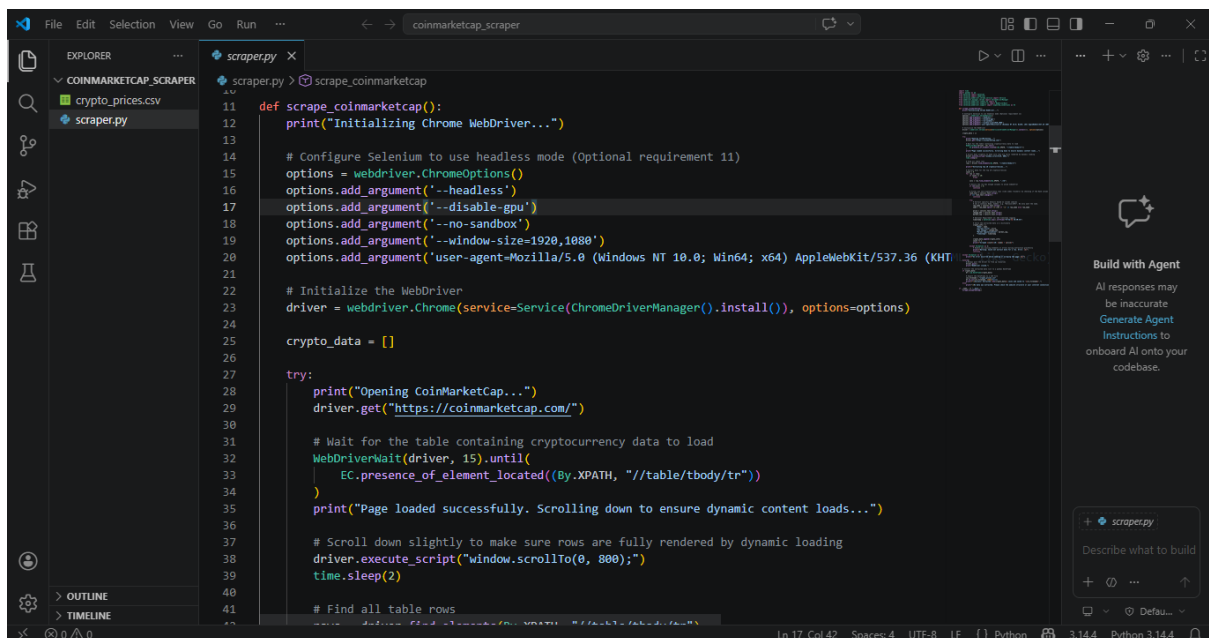
- Depends on website structure
- Requires internet connection
- Dynamic content may change frequently

11. Future Enhancements

- Track more than top 10 cryptocurrencies
- Add filtering options (e.g., highest gainers)
- Integrate with dashboards for visualization
- Schedule automatic data collection

12. Screenshots

Figure 1: Code Implementation



The screenshot displays a code editor with a dark theme. The Explorer panel on the left shows a project named 'COINMARKETCAP_SCRAPER' containing files 'crypto_prices.csv' and 'scraper.py'. The main editor window shows the 'scraper.py' file with the following Python code:

```
11 def scrape_coinmarketcap():
12     print("Initializing Chrome WebDriver...")
13
14     # Configure Selenium to use headless mode (Optional requirement 11)
15     options = webdriver.ChromeOptions()
16     options.add_argument('--headless')
17     options.add_argument('--disable-gpu')
18     options.add_argument('--no-sandbox')
19     options.add_argument('--window-size=1920,1080')
20     options.add_argument('user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML;
21
22     # Initialize the WebDriver
23     driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=options)
24
25     crypto_data = []
26
27     try:
28         print("Opening CoinMarketCap...")
29         driver.get("https://coinmarketcap.com/")
30
31         # Wait for the table containing cryptocurrency data to load
32         WebDriverWait(driver, 15).until(
33             EC.presence_of_element_located((By.XPATH, "//table/tbody/tr"))
34         )
35         print("Page loaded successfully. Scrolling down to ensure dynamic content loads...")
36
37         # Scroll down slightly to make sure rows are fully rendered by dynamic loading
38         driver.execute_script("window.scrollTo(0, 800);")
39         time.sleep(2)
40
41         # Find all table rows
```

The status bar at the bottom indicates the cursor is at line 17, column 42, in a UTF-8 file with 4 spaces, using LF line endings, in a Python 3.14.4 environment.

On the right side of the editor, there is a sidebar with a 'Build with Agent' section. It contains the text: 'All responses may be inaccurate. Generate Agent Instructions to onboard AI onto your codebase.' Below this, there is a search bar with the placeholder text 'Describe what to build' and a list of suggestions including 'scraper.py'.

```
File Edit Selection View Go Run ...
coinmarketcap_scraper

EXPLORER
COINMARKETCAP_SCRAPER
  crypto_prices.csv
  scraper.py

scraper.py
11 def scrape_coinmarketcap():
12     print("Initializing Chrome WebDriver...")
13
14     # Configure Selenium to use headless mode (Optional requirement 11)
15     options = webdriver.ChromeOptions()
16     options.add_argument('--headless')
17     options.add_argument('--disable-gpu')
18     options.add_argument('--no-sandbox')
19     options.add_argument('--window-size=1920,1080')
20     options.add_argument('user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML)')
21
22     # Initialize the WebDriver
23     driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=options)
24
25     crypto_data = []
26
27     try:
28         print("Opening CoinMarketCap...")
29         driver.get("https://coinmarketcap.com/")
30
31     except Exception as e:
32         print(f"Error: {e}")
33
34     driver.quit()
35
36     return crypto_data
37
38 if __name__ == '__main__':
39     data = scrape_coinmarketcap()
40     save_csv(data)
41
42     print("Script completed successfully.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Scraped 3/10: Tether - $1.00
Scraped 4/10: XRP - $1.40
Scraped 5/10: BNB - $620.63
Scraped 6/10: USD - $1.00
Scraped 7/10: Solana - $85.37
Scraped 8/10: TRON - $0.3231
Scraped 9/10: Dogecoin - $0.1098
Scraped 10/10: Hyperliquid - $40.58
WebDriver closed.

Success! Extracted 10 coins and saved to 'crypto_prices.csv'.
PS C:\Users\NITHISH\Desktop\cn\coinmarketcap_scraper>
```

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

scraper.py

Describe what to build

3.144 Python 3.14.4

AutoSave Off crypto_prices

File Home Insert Draw Page Layout Formulas Data Review View Automate Help

Clipboard Font Alignment Number Styles Cells Editing Sensitivity Add-ins

POSSIBLE DATA LOSS Some features might be lost if you save this workbook in the comma-delimited (.csv) format. To preserve these features, save it in an Excel file format. Don't show again Save As...

C16

	A	B	C	D	E	F	G	H	I	J	K
1	Name	Current Price	24h Change	Market Capitalization	Timestamp						
2	Bitcoin	\$77,589.24	1.28%	\$1,555,816,499,651	29-04-2026 16:18						
3	Ethereum	\$2,333.28	2.09%	\$282,187,927,817	29-04-2026 16:18						
4	Tether	\$1.00	0.02%	\$189,718,190,777	29-04-2026 16:18						
5	XRP	\$1.40	1.05%	\$86,491,042,875	29-04-2026 16:18						
6	BNB	\$628.63	0.90%	\$84,731,019,626	29-04-2026 16:18						
7	USDC	\$1.00	0.03%	\$77,582,350,909	29-04-2026 16:18						
8	Solana	\$85.37	1.94%	\$49,173,639,058	29-04-2026 16:18						
9	TRON	\$0.3231	0.16%	\$30,629,655,624	29-04-2026 16:18						
10	Dogecoin	\$0.1098	10.60%	\$16,918,320,937	29-04-2026 16:18						
11	Hyperliquid	\$40.58	1.44%	\$10,349,490,293	29-04-2026 16:18						
12											
13											
14											
15											
16											
17											
18											
19											
20											

Ready Accessibility: Unavailable

This project demonstrates how Selenium can be used to scrape real-time data from dynamic websites. It provides a practical understanding of web automation, data extraction, and storage. The project can be further extended for advanced analytics and financial applications.

14. Source Code

```
import time

import pandas as pd

from datetime import datetime

from selenium import webdriver

from selenium.webdriver.chrome.service import Service

from webdriver_manager.chrome import ChromeDriverManager

from selenium.webdriver.common.by import By

from selenium.webdriver.support.ui import WebDriverWait

from selenium.webdriver.support import expected_conditions as EC


def scrape_coinmarketcap():

    print("Initializing Chrome WebDriver...")


    # Configure Selenium to use headless mode (Optional requirement 11)
    options = webdriver.ChromeOptions()
    options.add_argument('--headless')
    options.add_argument('--disable-gpu')
    options.add_argument('--no-sandbox')
    options.add_argument('--window-size=1920,1080')
    options.add_argument('user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36')


    # Initialize the WebDriver
    driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=options)


    crypto_data = []


    try:

        print("Opening CoinMarketCap...")

        driver.get("https://coinmarketcap.com/")
```

```

# Wait for the table containing cryptocurrency data to load
WebDriverWait(driver, 15).until(
    EC.presence_of_element_located((By.XPATH, "//table/tbody/tr"))
)
print("Page loaded successfully. Scrolling down to ensure dynamic content loads...")

# Scroll down slightly to make sure rows are fully rendered by dynamic loading
driver.execute_script("window.scrollTo(0, 800);")
time.sleep(2)

# Find all table rows
rows = driver.find_elements(By.XPATH, "//table/tbody/tr")

print(f"Extracting top 10 cryptocurrencies...")

# Extract data for the top 10 cryptocurrencies
count = 0
for row in rows:
    if count >= 10:
        break

    cols = row.find_elements(By.XPATH, "./td")

    # Ensure the row has enough columns to avoid IndexError
    if len(cols) < 8:
        continue

    # Filter out non-cryptocurrency rows (like index trackers) by checking if the Rank column is a
    number
    rank_text = cols[1].text.strip()

```

```

if not rank_text.isdigit():
    continue

try:
    # Extract specific details based on column indices
    # Col 2 contains Name, Symbol. e.g. "Bitcoin\nBTC". We only want the name.
    raw_name = cols[2].text.strip()
    name = raw_name.split('\n')[0] if '\n' in raw_name else raw_name

    price = cols[3].text.strip()
    change_24h = cols[5].text.strip()
    market_cap = cols[7].text.strip()

    # Optional Requirement 12: Add timestamp logging
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # Store the extracted data in a dictionary
    crypto_info = {
        "Name": name,
        "Current Price": price,
        "24h Change": change_24h,
        "Market Capitalization": market_cap,
        "Timestamp": timestamp
    }

    crypto_data.append(crypto_info)
    count += 1
    print(f"Scraped {count}/10: {name} - {price}")

except Exception as e:
    # Handle any missing elements or errors during extraction gracefully

```

```
print(f"Warning: Could not extract data for a row. Error: {e}")
continue
```

```
except Exception as e:
```

```
    print(f"An error occurred while loading or scraping the page: {e}")
```

```
finally:
```

```
    # Always quit the driver to free up resources
```

```
    driver.quit()
```

```
    print("WebDriver closed.")
```

```
# Convert the extracted data list to a pandas DataFrame
```

```
if crypto_data:
```

```
    df = pd.DataFrame(crypto_data)
```

```
    # Export the DataFrame to a CSV file
```

```
    csv_filename = "crypto_prices.csv"
```

```
    df.to_csv(csv_filename, index=False)
```

```
    print(f"\nSuccess! Extracted {len(crypto_data)} coins and saved to '{csv_filename}'.")
```

```
else:
```

```
    print("\nNo data was extracted. Please check the website structure or your internet connection.")
```

```
if __name__ == "__main__":
```

```
    scrape_coinmarketcap()
```
